

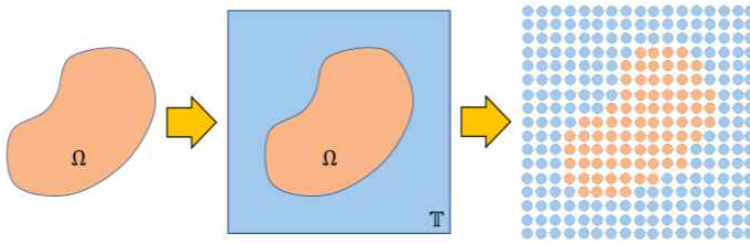
Domain Agnostic Fourier Neural Operators

02421001 Jeongeun Park

1. 서론

Fourier Neural Operator(FNO)는 PDE 해를 함수 공간에서 직접 학습할 수 있는 강력한 연산자 학습 모델이다. 그러나 기존 FNO는 직사각형 격자와 FFT에 의존하기 때문에 crack 이나 airfoil과 같은 불규칙한 domain을 직접 다루기 어렵다는 한계가 있다.

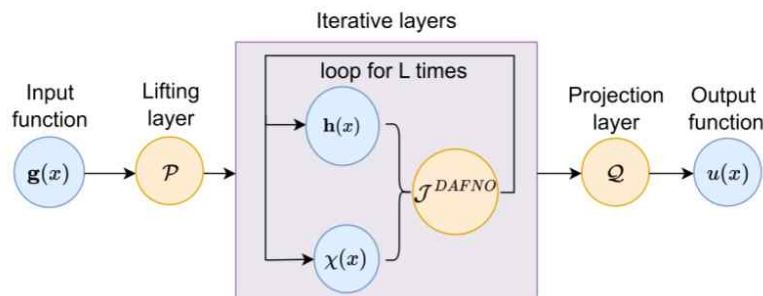
이를 해결하기 위해 본 논문에서는 Domain Agnostic Fourier Neural Operator (DAFNO)를 제안하였다. DAFNO의 핵심 아이디어는 domain 정보를 모델 구조 내부에 직접 포함시키는 것이다. domain을 나타내는 characteristic function을 Fourier convolution layer 내부 연산에 곱해줌으로써 domain 외부와의 비물리적 상호작용을 구조적으로 차단한다. 또한 sharp한 경계로 인해 발생하는 Gibbs 현상을 완화하기 위해 smoothed characteristic function을 사용한다. 이러한 작동 메커니즘은 논문의 그림을 통해 직관적으로 이해할 수 있다.



본 논문에서는 두 가지 변형을 제안한다. 바로 eDAFNO (explicit)와 iDAFNO (implicit)이다. eDAFNO의 경우 각 Fourier layer마다 독립적인 parameter를 사용하며, 상대적으로 높은 정확도를 보인다. iDAFNO의 경우 layer 간 parameter를 공유하여 parameter 수가 적고 학습이 안정적이다. 이번 과제에서는 이 중 eDAFNO 모델의 training code를 분석 대상으로 한다.

2. Training Code 개요

해당 training code는 airfoil 주변 유동 문제를 대상으로 하며, domain mask를 입력으로 받아 Mach number field를 예측하는 eDAFNO 모델을 학습한다. 전체적인 학습 흐름은 아래 그림과 같다.



먼저 airfoil domain 데이터를 로드한 후, domain characteristic function x 를 생성하고 smoothing을 적용한다. 이후 eDAFNO 모델을 정의하고, supervised learning 방식으로 학습을 수행한다. 마지막으로 L2 loss 기반 지표를 통해 모델 성능을 평가한다.

3. Training Code 분석

```
if __name__ == '__main__':
    # define hyperparameters
    lrs = [1e-2]
    gammas = [0.5]
    wds = [3e-6]
    smooth_coefs = [5.]
    smooth_coef = smooth_coefs[0]
    # experiments to be replicated with different seeds
    seeds = [5, 2000, 14000, 16000, 100000]
    seeds = [seeds[0]]

    #####
    #                                configs
    #####

    Ntotal = 1200
    ntrain = 1000
    ntest = 200

    batch_size = 20

    epochs = 501
    scheduler_step = 100
    tol_early_stop = 500

    modes = 12
    width = 32
    nlayers = 4
```

training code의 시작 부분에서는 핵심 hyper parameter들이 정의된다. 이후 학습에 필요한 입력 데이터들이 로드되며, mask, dist, chi, 그리고 x , y grid가 사용된다.

```

t1 = default_timer()

sub = 1
Sx = int(((101 - 1) / sub) + 1)
Sy = Sx

mask = np.load('./data_airfoil_naca/NACA_mask_interp.npy')
mask = torch.tensor(mask, dtype=torch.float)
dist_in = np.load('./data_airfoil_naca/NACA_dist.npy')
dist_in = torch.tensor(dist_in[:Ntotal, :, :], dtype=torch.float)
input = smooth_chi(mask, dist_in, smooth_coef)
# input = mask
output = np.load('./data_airfoil_naca/NACA_Q_interp.npy')
output = torch.tensor(output, dtype=torch.float)

mask_train = mask[:Ntotal][:ntrain, ::sub, ::sub][:, :Sx, :Sy]
mask_test = mask[:Ntotal][-ntest:, ::sub, ::sub][:, :Sx, :Sy]

mask_train = mask_train.reshape(ntrain, Sx, Sy, 1)
mask_test = mask_test.reshape(ntest, Sx, Sy, 1)

chi_train = input[:Ntotal][:ntrain, ::sub, ::sub][:, :Sx, :Sy]
chi_test = input[:Ntotal][-ntest:, ::sub, ::sub][:, :Sx, :Sy]

chi_train = chi_train.reshape(ntrain, Sx, Sy, 1)
chi_test = chi_test.reshape(ntest, Sx, Sy, 1)

y_train = output[:Ntotal][:ntrain, ::sub, ::sub][:, :Sx, :Sy]
y_test = output[:Ntotal][-ntest:, ::sub, ::sub][:, :Sx, :Sy]

```

각 데이터의 구성은 다음과 같다. mask는 airfoil domain의 boundary mask, dist는 그 boundary로부터의 거리 정보, chi는 smoothed characteristic function, 마지막으로 x, y grid는 공간 좌표 정보를 의미한다. 최종적으로 모델의 입력은 (batch, x, y, 1) 형태의 x 필드로 구성된다.

```

y_train = output[:Ntotal][:ntrain, ::sub, ::sub][:, :Sx, :Sy]
y_test = output[:Ntotal][-ntest:, ::sub, ::sub][:, :Sx, :Sy]

# Normalizers
y_normalizer = GaussianNormalizer(y_train)
y_train = y_normalizer.encode(y_train)

y_train = y_train.reshape(ntrain, Sx, Sy, 1)
y_test = y_test.reshape(ntest, Sx, Sy, 1)

```

출력 데이터는 위와 같으며, y는 Mach number field이다. 학습 전에 Gaussian Normalizer를 통해 정규화된다.

4. 모델 구조 분석 (eDAFNO2d)

```
class eDAFNO2d(nn.Module):  
    def forward(self, chi):  
        batchsize = chi.shape[0]  
        size_x = size_y = chi.shape[1]  
  
        grid = self.get_grid(batchsize, size_x, size_y, chi.device)  
  
        chi = chi.permute(0, 3, 1, 2)  
        chi = chi.expand(batchsize, self.width, size_x, size_y)  
        x = grid  
  
        # Lifting layer:  
        x = self.fc0(x)  
        x = x.permute(0, 3, 1, 2)  
  
        for i in range(self.nlayers):  
            conv_chi = self._modules['conv%d' % i](chi)  
            conv_chix = self._modules['conv%d' % i](chi * x)  
            xconv_chi = x * conv_chi  
            wx = self._modules['w%d' % i](x)  
            x = chi * (conv_chix - xconv_chi + wx)  
            if i < self.nlayers - 1: x = F.gelu(x)  
  
        x = x.permute(0, 2, 3, 1)  
  
        # Projection Layer:  
        x = self.fc1(x)  
  
        return x
```

위 code block에서 보듯이 eDAFNO2d는 Lifting layer (P) -> Fourier integral layers (L layers) -> Projection layer (Q)로 구성된다.

이때 Lifting Layer에서는 입력 필드 x 와 공간 좌표 정보를 받아 고차원 characteristic 공간으로 매핑한다. 이후의 Fourier Integral Layer는 DAFNO의 핵심 연산이 수행되는 구간이다.

```
x = chi * (conv_chix - xconv_chi + wx)
```

characteristic function x 가 커널 연산의 integrand에 직접 곱해짐으로써 domain 외부와의 비물리적인 상호작용을 차단한다. 특히 FFT 연산 과정에서 발생하는 Gibbs 현상을 방지하기 위해 smoothed characteristic function x 를 활용하는 구조를 갖는다. 이를 통해 불규칙한 domain에서도 물리적으로 타당한 예측이 가능해진다.

5. Training Loop 분석

```
for ep in range(epochs):
    t1 = default_timer()
    optimizer = scheduler(optimizer,
                           LR_schedule(learning_rate, ep, scheduler_step, scheduler_gamma))
    model.train()
    train_l2 = 0
    for mm, xx, yy in train_loader:
        mm, xx, yy = mm.to(device), xx.to(device), yy.to(device)

        optimizer.zero_grad()
        out = model(xx)
        out = y_normalizer.decode(out) * mm
        yy = y_normalizer.decode(yy) * mm

        loss = myloss(out, yy)
        train_l2 += loss.item()

    loss.backward()
    optimizer.step()
```

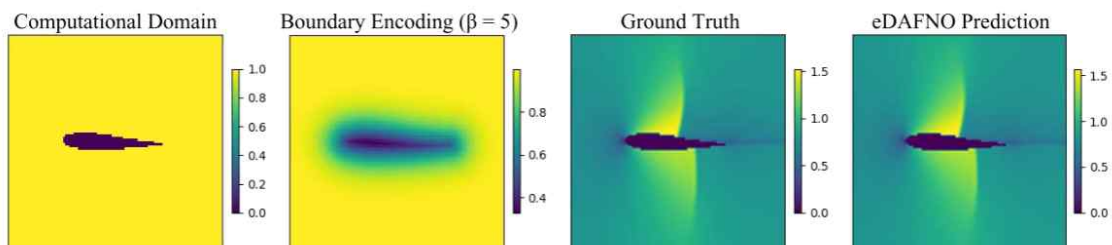
Training loop에서는 입력 x 필드에 대해 forward pass를 수행한 후, 예측된 Mach field와 실제 데이터 사이의 오차를 Relative L2 loss를 통해 계산한다. 마지막으로 Adam optimizer를 통해 모델 parameter를 업데이트한다.

6. 실험 결과 및 성능 분석

분석한 eDAFNO 모델의 성능을 검증하기 위해, 논문에서 제시된 airfoil design 데이터셋에 대한 실험 결과를 정리하였다. 해당 실험은 NACA-0012 airfoil 주변의 초음속 유동을 시뮬레이션한 데이터를 기반으로 하며, 모델은 airfoil의 mask를 입력받아 주변의 Mach number 필드를 예측했다.

	Model	Train error	Test error
Proposed model	eDAFNO	0.329%±0.020%	0.596%±0.005%
	iDAFNO	0.448%±0.012%	0.642%±0.020%
	eDAFNO on irregular grids	0.331%±0.003%	0.659%±0.007%
Baseline model	Geo-FNO	1.565%±0.180%	1.650%±0.175%
	F-FNO	0.566%±0.066%	0.794%±0.025%
	FNO w/ mask	2.676%±0.054%	3.725%±0.108%
	UNet w/ mask	2.781%±1.084%	4.957%±0.059%

실험 결과 eDAFNO는 기존의 Neural Operator 모델들보다 압도적으로 낮은 오차율을 기록하였다. 이는 단순히 입력에 마스크를 추가하는 것보다, DAFNO처럼 연산자 구조 내부에 도메인 정보를 인코딩하는 방식이 훨씬 효과적임을 입증한다.



모델의 예측 성능은 시각적으로도 확인할 수 있다. 그림에 나타난 것처럼 eDAFNO는 smoothed characteristic function x 를 사용함으로써 airfoil의 날카로운 꼬리 부분에서 발생할 수 있는 Gibbs 현상을 효과적으로 억제하였으며, 복잡한 경계면 근처의 물리량을 정밀하게 모사하였다.

7. 결론 및 시사점

본 training code가 구현하는 eDAFNO 모델은 domain 정보를 Fourier integral layer에 직접 통합함으로써 기존 FNO의 한계였던 불규칙 domain 문제를 해결함과 동시에 높은 정확도를 확보하였다. 특히 eDAFNO는 iDAFNO 대비 parameter 수는 많지만 복잡한 유체 역학 문제에서 더 높은 표현력을 바탕으로 최적의 성능을 낼 수 있음을 실험적으로 확인하였다. 이러한 특성은 향후 복잡한 형상의 설계 최적화나 실시간 물리 시뮬레이션 분야에서 eDAFNO가 강력한 surrogate model로 활용될 수 있음을 시사한다.